



32-bit Pipelined MIPS Processor Design

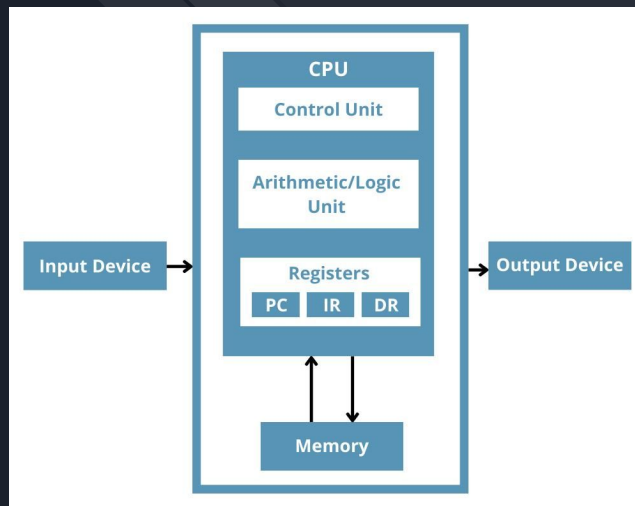
By: Matthew Bernardino, Minh Dinh, and
Shaan Lele

ECPE 136

April 22 2026

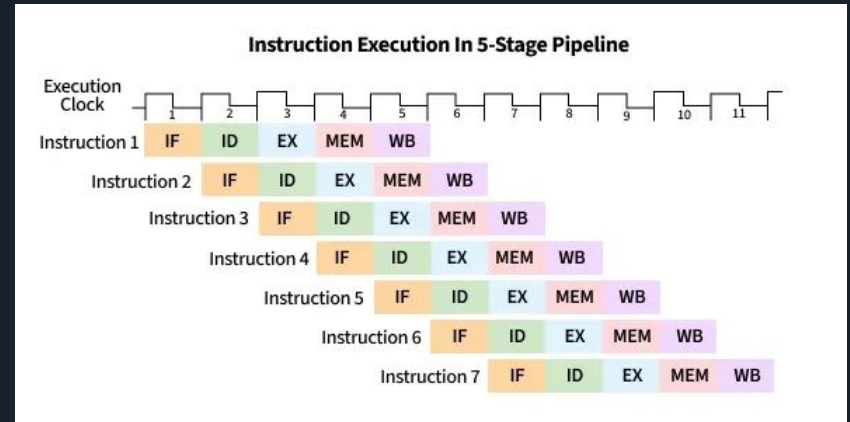
Project Overview

- 32-bit MIPS processor designed in SystemVerilog
- Based on RISC architecture (simple instructions)
- Supports arithmetic, memory, and branch operations
- Built using modular hardware components
- Simulated and verified using HDL tools



Pipelining Concept

- Improves performance by overlapping instructions
- Multiple instructions processed at the same time
- Divides execution into stages:
 - IF - Instruction Fetch
 - ID - Instruction Decode
 - EX - Execute
 - MEM - Memory Access
 - WB - Write Back
- Works like an assembly line

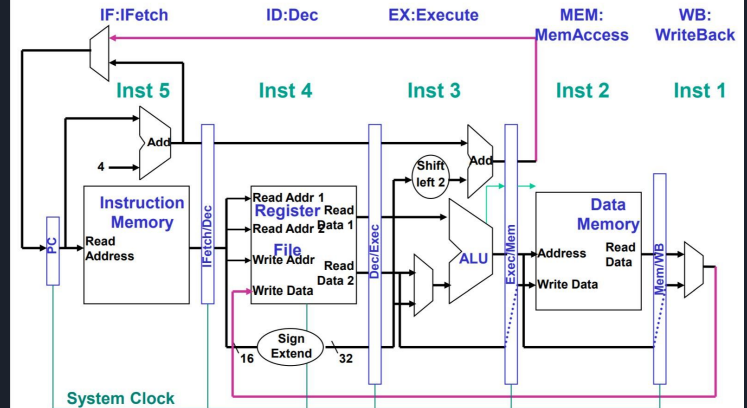


Processor Architecture

- Datapath performs computations and data movement
- Control unit generates signals based on instruction type
- Pipeline registers separate each stage
- Instruction and data memory used for operation
- Hazard handling ensures correct execution (forwarding, stalling, flushing)

MIPS Pipelined Datapath

- **State registers** between pipeline stages to **isolate** them



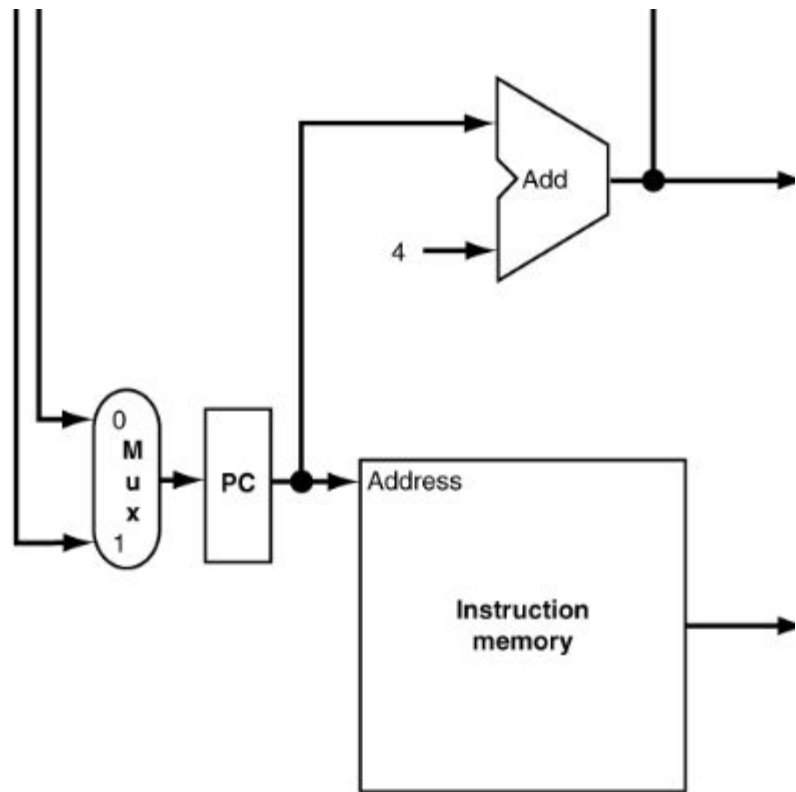

$$PC + (PC + 4)$$

What It Does:

- Stores current instruction address (PC)
- Increments PC by 4 every cycle
- Sends address to instruction memory

How is it important?

- Drives the entire pipeline forward
- Incorrect PC = completely wrong execution
- Needs to work with hazards (stall/flush)



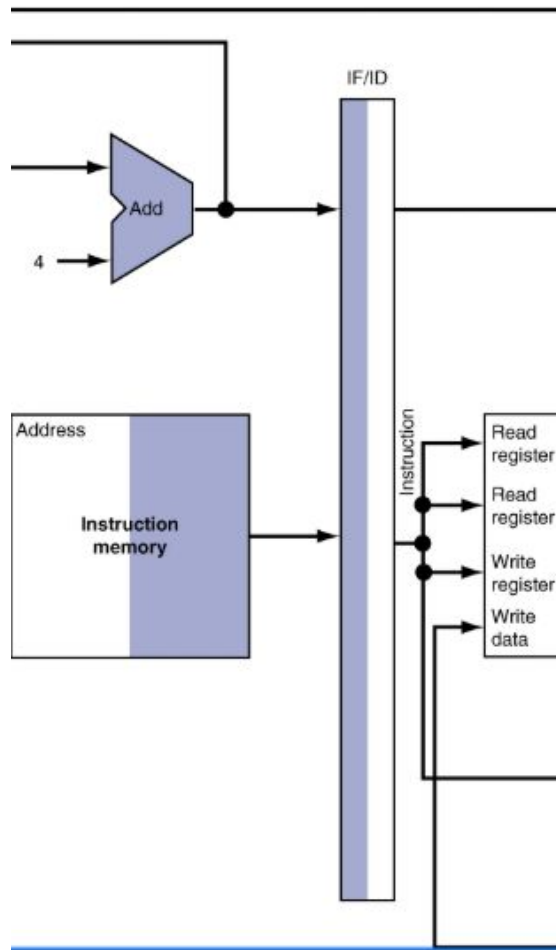
IF/ID Pipeline Register

What It Does:

- Stores instruction + PC+4 between IF and ID
- Updates on clock edge
- Holds values during stalls

How is it important?

- Separates pipeline stages cleanly
- Prevents timing issues between IF and ID
- Critical for hazard handling



Decode Stage: Control Unit

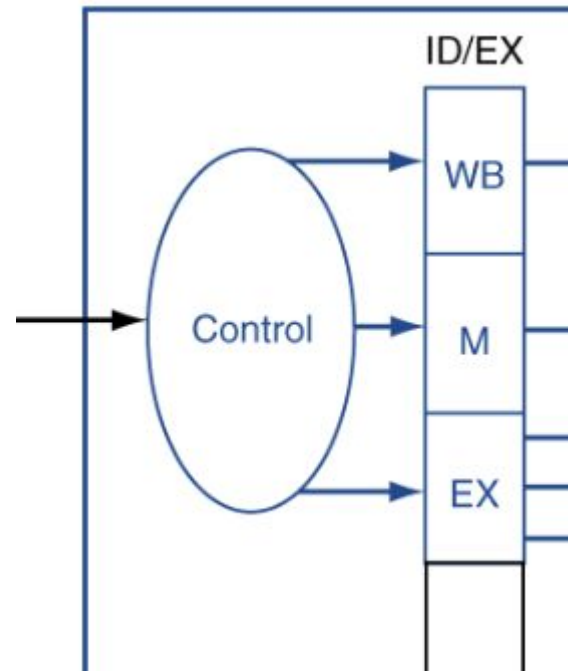
What It Does:

- Reads the 6-bit opcode
- Figures out the instruction will be executed.
- Makes the control signals for the data path.

How is it important?

- Tells the processor what path the instruction needs to take.
- Tells the datapath what components need to be used.

R-type	<table border="1"><tr><td>op</td><td>rs</td><td>rt</td><td>rd</td><td>shamt</td><td>funct</td></tr></table>						op	rs	rt	rd	shamt	funct	ADD, SUB, AND, OR
op	rs	rt	rd	shamt	funct								
	Arithmetic instruction format												
I-type	<table border="1"><tr><td>op</td><td>rs</td><td>rt</td><td>address/immediate</td></tr></table>				op	rs	rt	address/immediate	SW, BEQ, LW, ADDI, ORI				
op	rs	rt	address/immediate										
	Transfer, branch, immediate.												
J-type	<table border="1"><tr><td>op</td><td>target address</td></tr></table>		op	target address	J loop								
op	target address												
	Jump instruction												
Field size	<table border="1"><tr><td>6 bits</td><td>5bits</td><td>5bits</td><td>5bits</td><td>5bits</td><td>6 bits</td></tr></table>	6 bits	5bits	5bits	5bits	5bits	6 bits						
6 bits	5bits	5bits	5bits	5bits	6 bits								



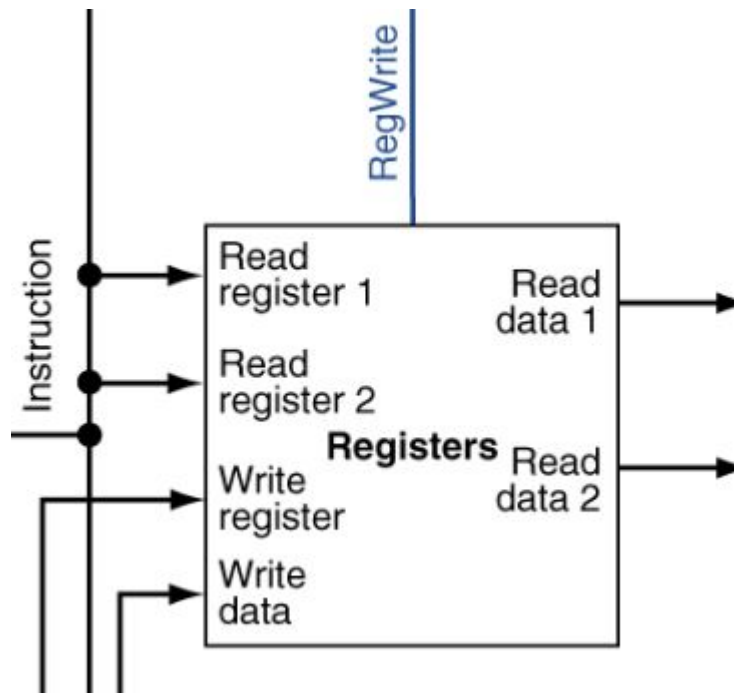
Decode Stage: Register File

What It Does:

- Stores 32 MIPS registers.
- Reads two register values rs and rt.
- Will write back a value if there is a destination register.

How is it important?

- Most instructions need data from registers.
- Gets the data from the registers and pushes it through the pipeline.



Decode Stage: Immediate Handle

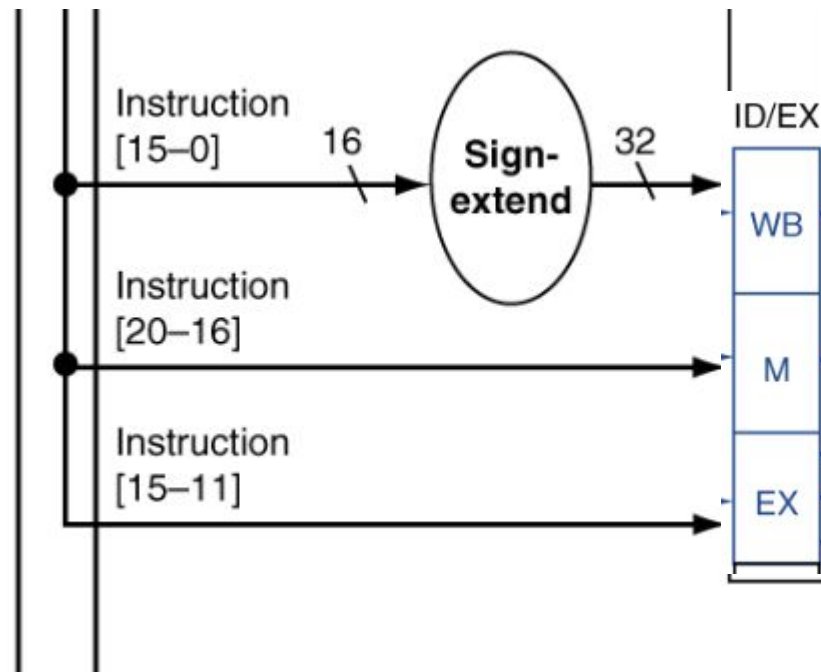
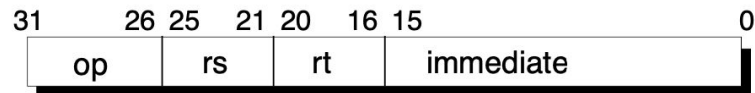
What It Does:

- Takes a 16 bit immediate from instruction.
- Extends it to 32 bits
- Ex Sign Ext: lw, sw, beq, addi
- Ex Zero Ext: ori

How is it important?

- ALU is 32 bits
- Immediate only give 16 bits
- Makes immediate usable within the datapath
- Only I-type instructions

I-Type (Immediate)



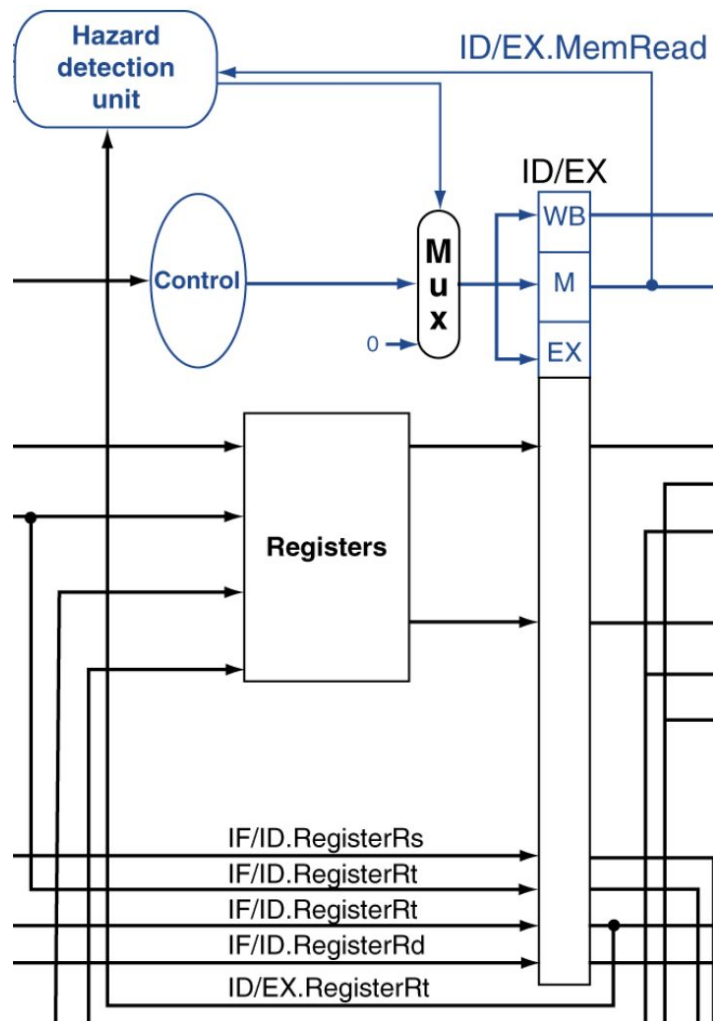
In Between Stage: Decode / Execute Handoff

What It Does:

- Stores control signals and data values from decode stage.
- Stores on clock edge
- Passes these values into the execute stage on the next clock.

How is it important?

- Separates the decode and execute stages.
- Keeps pipeline stages independent and efficient.
- Accounts for stall and clear when hazard handling.



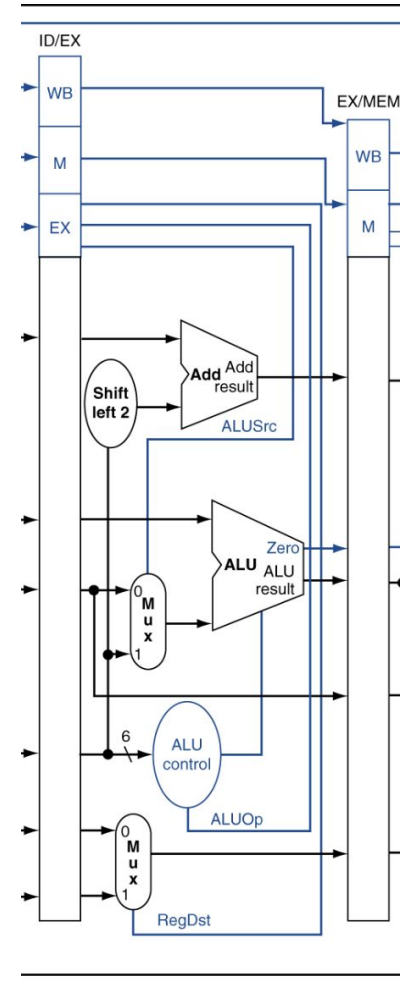
Execute Stage: ALU Control

What It Does:

- Receives the category operation from the control unit.
- If R-Type = check funct field. (add, sub, and, or).
- Creates exact ALU control signal.

How is it important?

- Translates general information into exact ALU operation.
- Independent and efficient designing for making ALU simple.



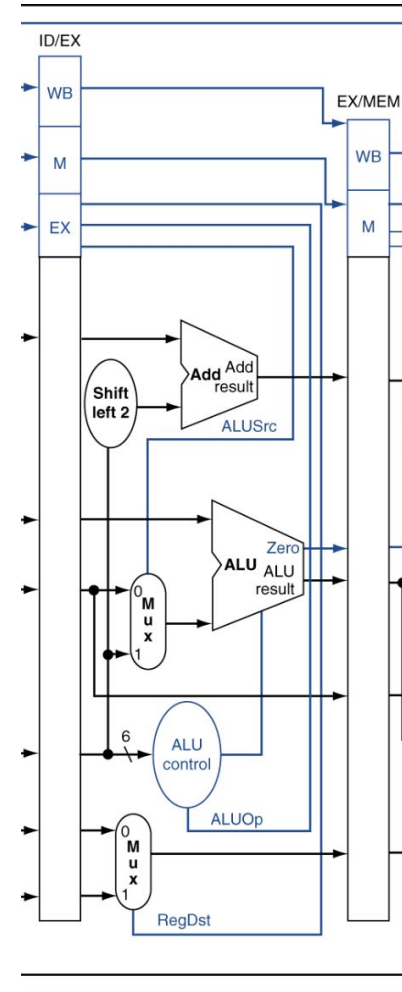
Execute Stage: ALU

What It Does:

- Performs actual arithmetic and logic operation.
- Inputs = two 32 bit values & Control signals
- Outputs result and check.

How is it important?

- Operations: And, Or, Add, Sub, & NOP
- Main computation actually happens here.
- Comparison for beq and address calculation.



Instruction Decode: Hazard Detection Unit

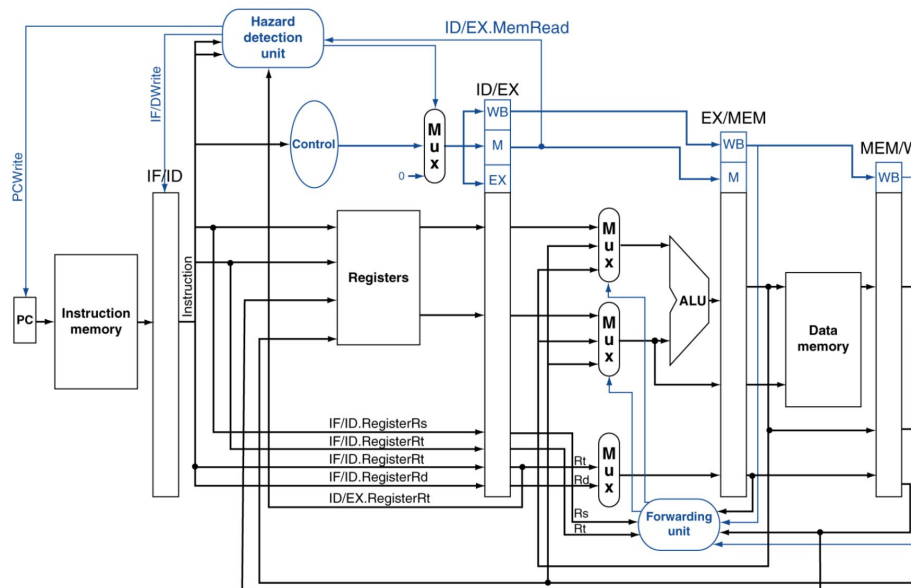
What It Does:

- It watches for “Load-Use” conflicts where even forwarding isn't fast enough to provide the data.

How is it important?

- If an instruction needs data from a LW (Load Word) command immediately, this module tells the CPU to “freeze” for one cycle.
- It is the safety net that prevents the processor from using outdated data

Datapath with Hazard Detection



MK
MICHAEL KAPLAN

Chapter 4 — The Processor

Instruction 1: LW \$5, 100(\$0) (Read data from the Filing Cabinet into pocket \$5)

Instruction 2: ADD \$7, \$5, \$6 (Immediately try to do math using pocket \$5)

Execute Stage: Forwarding Unit

What It Does:

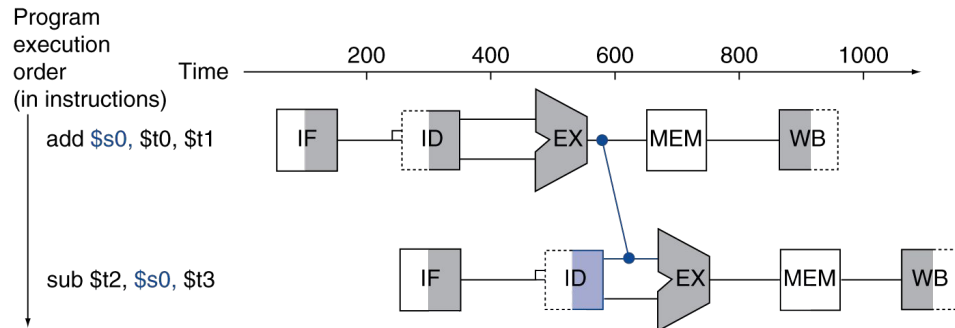
- It intercepts a math answer from a later stage and passes it directly back to the ALU.
- It fixes “Data Hazards” where Instruction B needs an answer that Instruction A hasn't finished saving to the registers yet.

How is it important?

- It allows the CPU to keep running at full speed without having to wait (stall) for the Register File to update.

Data Hazards

- An instruction depends on completion of data access by a previous instruction
 - add **\$s0**, \$t0, \$t1
 - sub \$t2, **\$s0**, \$t3



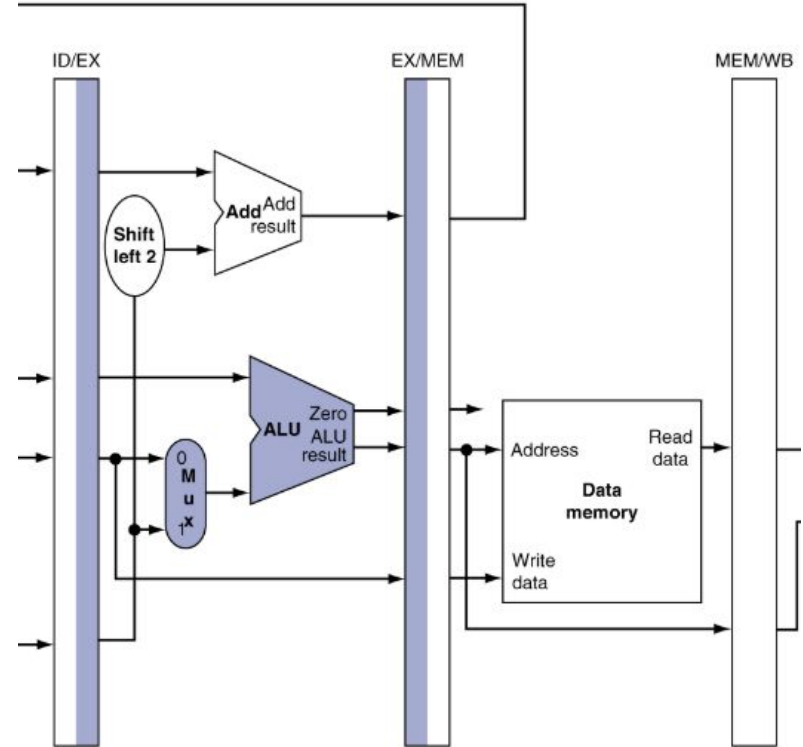
Sits between EX and MEM Stage: EX/MEM Register

What It Does:

- It holds the ALU's math results for the next stage.
- It ensures that “Store” commands have a stable address and stable data to work with when talking to the memory for one clock cycle.

How is it important?

- Without it, the ALU could start its next instruction and overwrite the current results before the Memory stage could store them.



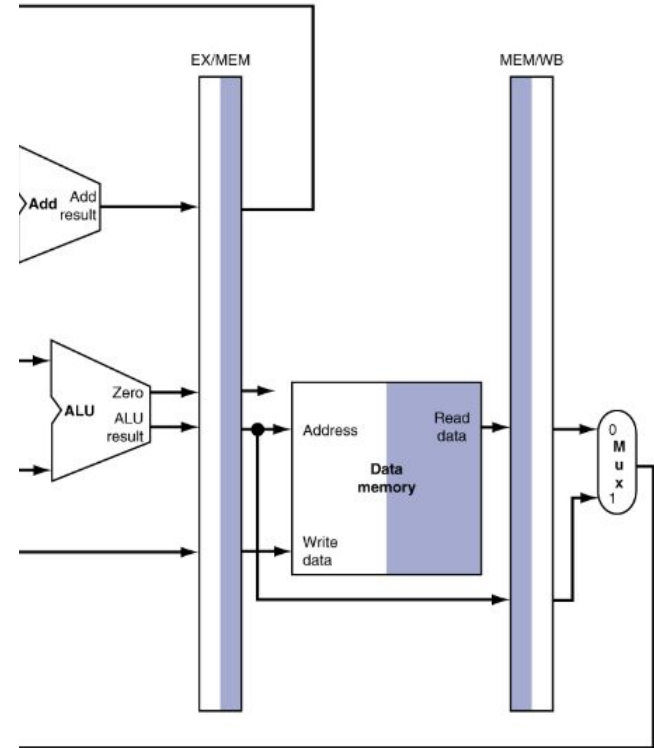
MEM Access Stage: Data Memory

What It Does:

- It is the processor's RAM to store long term information.
- LW (Load Word) can load data to registers and SW (Store Word) can save data into memory for later use.

How is it important?

- Registers are fast, they are extremely small. Memory allows the processor to manage more data.



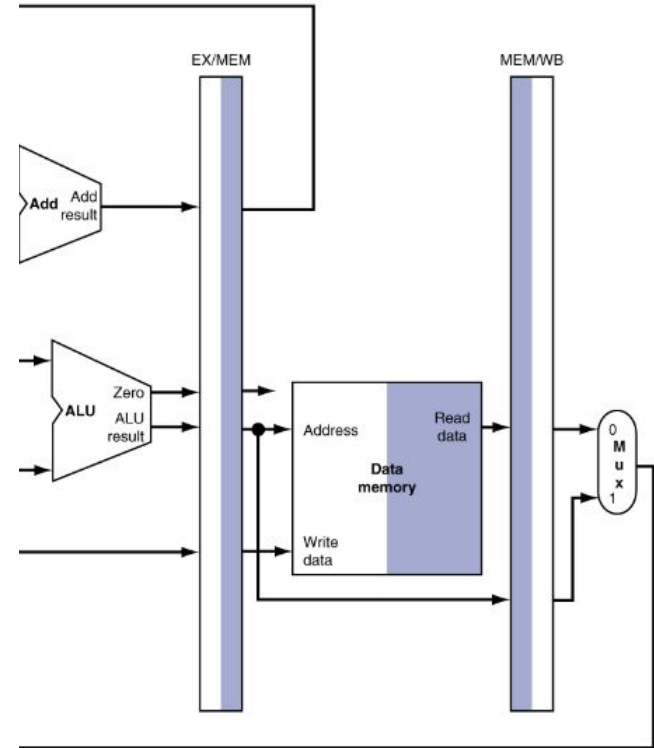
Sits between MEM and WB Stage: MEM/WB Register

What It Does:

- It holds the data coming out of the memory for the final stage.
- It prevents the processor from losing the data it just fetched from the memory while it prepares for the final WB operation.

How is it important?

- Allows the memory stage to start looking for the next instruction's data while the current instruction is saved for the final stage.



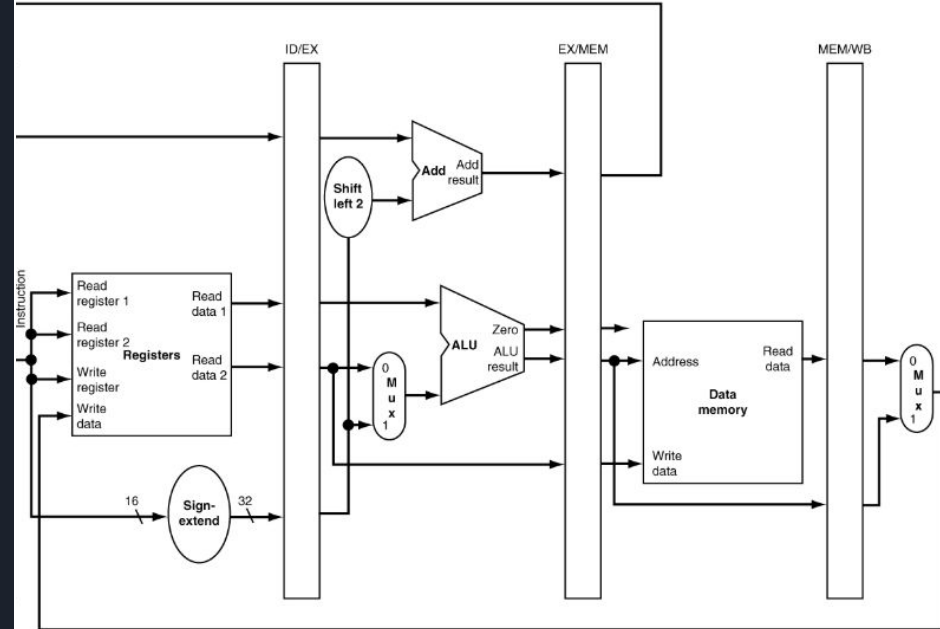
WB Stage: Writeback Mux

What It Does:

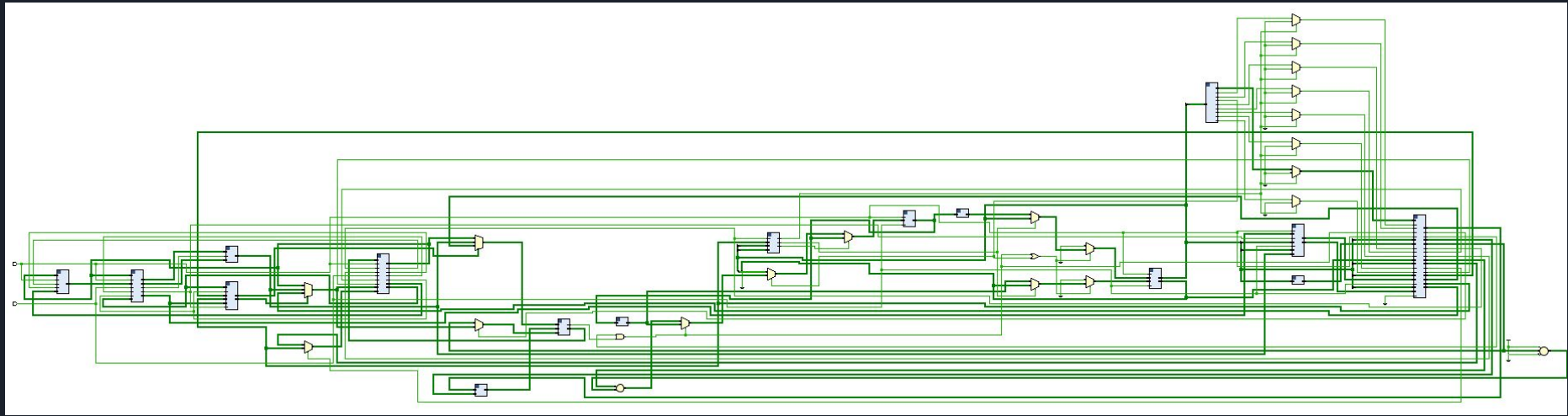
- It's a multiplexer that chooses between the ALU math result or memory result (LW) to save back into the register.

How is it important?

- It tells the processor the correct result to save into the register.



RTL Diagram



Simulation Results

Time=	0	PC= 0	Pipeline Filling / NOP		\$t0=x	\$t1=x	\$t2=x	\$t3=x	\$t4=x	\$t5=x	\$t6=x	\$t7=x	\$t8=x	\$t9=x
Time=	5000	PC= 0	Pipeline Filling / NOP		\$t0= 0	\$t1= 0	\$t2= 0	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	25000	PC=4	Pipeline Filling / NOP		\$t0= 0	\$t1= 0	\$t2= 0	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	35000	PC=8	Pipeline Filling / NOP		\$t0= 0	\$t1= 0	\$t2= 0	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	45000	PC=12	Pipeline Filling / NOP		\$t0= 0	\$t1= 0	\$t2= 0	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	55000	PC=16	addi \$t0, \$zero, 5		\$t0= 0	\$t1= 0	\$t2= 0	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	65000	PC=20	ori \$t1, \$zero, 3		\$t0= 5	\$t1= 0	\$t2= 0	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	75000	PC=24	add \$t2, \$t0, \$t1		\$t0= 5	\$t1= 3	\$t2= 0	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	85000	PC=28	sub \$t3, \$t0, \$t1		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 0	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	95000	PC=32	and \$t4, \$t0, \$t1		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 2	\$t4= 0	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	105000	PC=36	or \$t5, \$t0, \$t1		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 2	\$t4= 1	\$t5= 0	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	115000	PC=36	or \$t5, \$t0, \$t1		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 2	\$t4= 1	\$t5= 7	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	125000	PC=40	sw \$t2, 0(\$zero)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 2	\$t4= 1	\$t5= 7	\$t6= 0	\$t7= 0	\$t8= 0	\$t9= 0
Time=	135000	PC=44	lw \$t6, 0(\$zero)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 2	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 0	\$t8= 0	\$t9= 0
Time=	145000	PC=48	add \$t7, \$t6, \$t1		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 2	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 0	\$t8= 0	\$t9= 0
Time=	155000	PC=52	addi \$t3, \$zero, -5		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 2	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 0	\$t9= 0
Time=	165000	PC=56	beq \$t2, \$t6, 2		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 0	\$t9= 0
Time=	175000	PC=60	addi \$t0, \$zero, 99 (Skipped)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 0	\$t9= 0
Time=	185000	PC=64	addi \$t1, \$zero, 99 (Skipped)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 0	\$t9= 0
Time=	195000	PC=68	j 15 (Jump to PC 60)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 0	\$t9= 0
Time=	205000	PC=72	addi \$t2, \$zero, 99 (Skipped)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 0	\$t9= 0
Time=	215000	PC=68	add \$t8, \$t0, \$t1		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 0	\$t9= 0
Time=	225000	PC=72	sub \$t9, \$t8, \$t0		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 8	\$t9= 0
Time=	235000	PC=68	j 17 (Infinite Loop)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 8	\$t9= 3
Time=	245000	PC=72	j 17 (Infinite Loop)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 8	\$t9= 3
Time=	255000	PC=68	j 17 (Infinite Loop)		\$t0= 5	\$t1= 3	\$t2= 8	\$t3= 5	\$t4= 1	\$t5= 7	\$t6= 8	\$t7= 11	\$t8= 8	\$t9= 3

```

module instr_mem (
    input  logic [31:0] addr,
    output logic [31:0] instr
);

    logic [31:0] mem [0:255];

    initial begin
        // Initialize data
        mem[0] = 32'h20080005; // addi $t0, $zero, 5 (Put 5 in $t0 - Reg 8)
        mem[1] = 32'h34090003; // ori  $t1, $zero, 3 (Put 3 in $t1 - Reg 9)

        // Test math
        mem[2] = 32'h01095020; // add  $t2, $t0, $t1 ($t2 (Reg 10) = 5 + 3 = 8)
        mem[3] = 32'h01095822; // sub  $t3, $t0, $t1 ($t3 (Reg 11) = 5 - 3 = 2)
        mem[4] = 32'h01096024; // and  $t4, $t0, $t1 ($t4 (Reg 12) = 5 AND 3 = 1)
        mem[5] = 32'h01096825; // or   $t5, $t0, $t1 ($t5 (Reg 13) = 5 OR 3 = 7)

        // Test memory and stalling for "load-use" hazard
        mem[6] = 32'hA0A00000; // sw   $t2, 0($zero) (Store 8 into memory address 0)
        mem[7] = 32'h8C0E0000; // lw   $t6, 0($zero) (Load from memory address 0 into $t6 - Reg 14 = 8)
        mem[8] = 32'h01C97820; // add  $t7, $t6, $t1 ($t7 (Reg 15) = 8 + 3 = 11) - Testing hazard unit

        // Test negative number
        mem[9] = 32'h200BFFFB; // addi $t3, $zero, -5 (Put -5 in $t3 - Reg 11)

        // Test branch and jump (Shifted down by 1)
        mem[10] = 32'h114E0002; // beq  $t2, $t6, 2 (If $t2(8) == $t6(8), skip the next 2 instructions)
        mem[11] = 32'h20080063; // addi $t0, $zero, 99 (Should be skipped)
        mem[12] = 32'h20090063; // addi $t1, $zero, 99 (Should be skipped)

        // Jump the next instruction
        mem[13] = 32'h0800000F; // j    address 15 (Jump past the next instruction)
        mem[14] = 32'h200A0063; // addi $t2, $zero, 99 (Should be skipped)

        // Test forwarding module
        mem[15] = 32'h0109C020; // add  $t8, $t0, $t1 ($t8 (Reg 24) = 5 + 3 = 8)
        mem[16] = 32'h0308C822; // sub  $t9, $t8, $t0 ($t9 (Reg 25) = 8 - 5 = 3)

        // End of test
        mem[17] = 32'h08000011; // j    address 17 (Loop to stop the program)

        // Initialize extra remaining memory slots to 0
        for (integer i = 18; i < 32; i++) begin
            mem[i] = 32'd0;
        end
    end

    // Word align the address (addr[31:2])
    assign instr = mem[addr[31:2]];

endmodule

```

References

J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017. A

T. J. Sparks, *Pipelined MIPS Processor*, GitHub repository

Dr. Balasubramaniyan Chandrasekaran

